



Machine Learning Methods

Lecture 13 Genetic Algorithm

Answering the AI Question

- How do we get an agent to act intelligently?
- One answer:
 - Humans have evolved a brain
 - Which enables us to act intelligently
- Idea:
 - Mimic Darwinian evolution to evolve intelligent agents
- Intelligence is a specific instance of evolution
 - Evolving a species to survive in a niche
 - Via adaptation of species through survival of the fittest
- More general idea:
 - Evolve programs to solve problems

Evolutionary Approaches to AI

- John Holland 1975
 - “Adaptation in Natural and Artificial Systems”
- Developed the idea of a genetic algorithm
 - Searching via sampling hyperplane partitions of the search space
 - Still just search over a space
- Broader sense:
 - Somehow representing solutions to a problem
 - And allowing solutions to be combined into new ones
 - And testing whether the new ones are improvements

Evolutionary Computation

- Computational procedures patterned after biological evolution.
- Search procedure that probabilistically applies search operators to set of points in the search space
- Evolutionary Algorithm, Evolutionary Programming, Genetic Algorithm & Genetic Programming.

Evolution ...

Lamarck and others:

- Species “transmute” over time

Darwin and Wallace:

- Consistent, heritable variation among individuals in population
- Natural selection of the fittest

Mendel and genetics:

- A mechanism for inheriting traits
- genotype → phenotype mapping

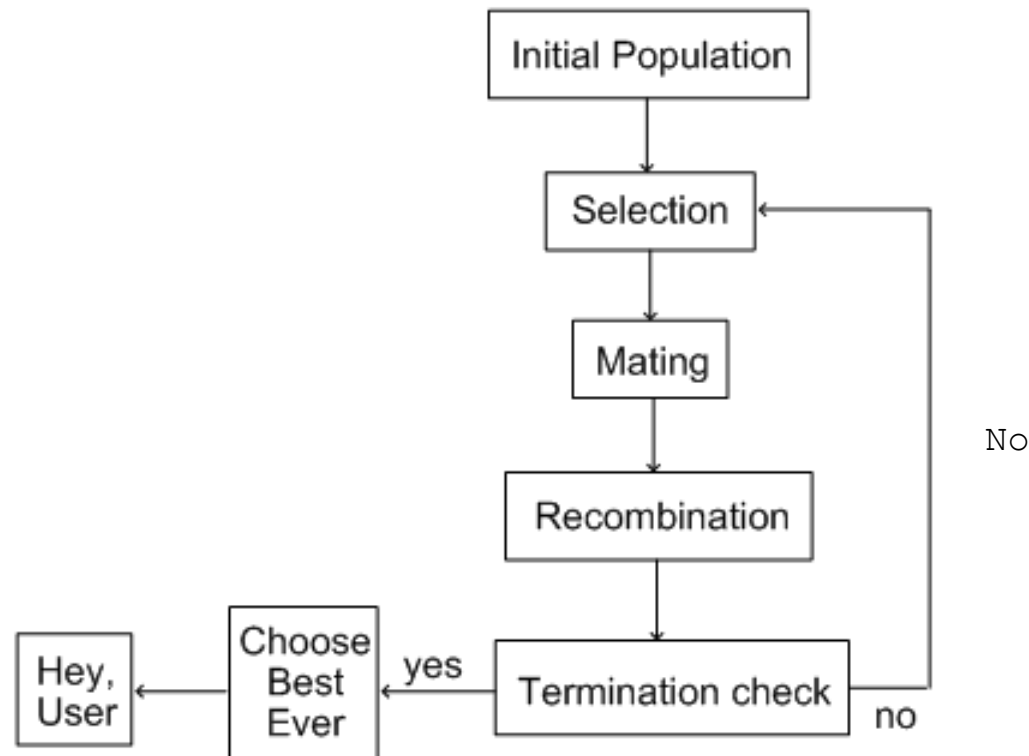
Genetic Algorithms (GAs) and Genetic Programming (GP)

- Genetic Algorithms
 - Optimising parameters for problem solving
 - Fix a format for the parameters in a problem solution
 - Represent the parameters in the solution(s)
 - As a “bit” string normally, but often something else
 - Evolve answers in this representation
- Genetic Programming
 - Representation of solutions is richer in general
 - Solutions can be interpreted as programs
 - Evolutionary process is very similar
 - Covered in next lecture

Overview of Classical GAs

- Representation of parameters is a bit string
 - Solutions to a problem represented in binary
 - 101010010011101010101
 - Everything in computers is represented in binary
- Start with a population (fairly large set)
 - Of possible solutions known as **individuals**
- Combine possible solutions by swapping material
 - Choose the “best” solutions to swap material between and kill off the worse solutions
 - This generates a new set of possible solutions
- Requires a notion of “fitness” of the individual
 - Base on an evaluation function with respect to the problem
 - Dependent on properties of the solution

The Canonical Genetic Algorithm



GA(*Fitness*, *Fitness_threshold*, *p*, *r*, *m*)

- *Initialize*: $P \leftarrow p$ random hypotheses
- *Evaluate*: for each h in P , compute $Fitness(h)$
- While $[\max_h Fitness(h)] < Fitness_threshold$
 1. *Select*: Probabilistically select $(1 - r)p$ members of P to add to P_s .

$$\Pr(h_i) = \frac{Fitness(h_i)}{\sum_{j=1}^p Fitness(h_j)}$$

2. *Crossover*: Probabilistically select $\frac{r \cdot p}{2}$ pairs of hypotheses from P . For each pair, $\langle h_1, h_2 \rangle$, produce two offspring by applying the Crossover operator. Add all offspring to P_s .
 3. *Mutate*: Invert a randomly selected bit in $m \cdot p$ random members of P_s
 4. *Update*: $P \leftarrow P_s$
 5. *Evaluate*: for each h in P , compute $Fitness(h)$
- Return the hypothesis from P that has the highest fitness.



1.)

The Initial Population

- Represent solutions to problems
 - As a bit string of length L
 - (Difficult part of using GA's – see later)
- Choose an initial population size
 - Generate length L strings of 1s & 0s randomly
 - Sometimes (rarely) done more intelligently
- Strings are sometimes called chromosomes
 - Letters in the string are called “genes”
 - See later for the (bad) analogies
 - We call the bit-string “individuals”

Represent

$(Outlook = Overcast \vee Rain) \wedge (Wind = Strong)$

by

<i>Outlook</i>	<i>Wind</i>
011	10

Represent

IF $Wind = Strong$ THEN $PlayTennis = yes$

by

<i>Outlook</i>	<i>Wind</i>	<i>PlayTennis</i>
111	10	10

The Selection Process

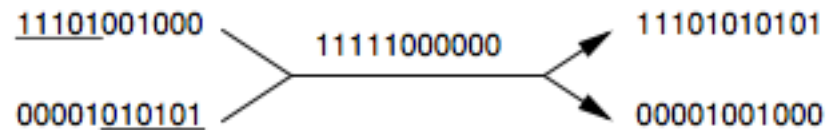
- We are going to combine pairs of members of the current population to produce offspring
 - So, we need to choose which pairs to combine

To choose which pair to combine, we

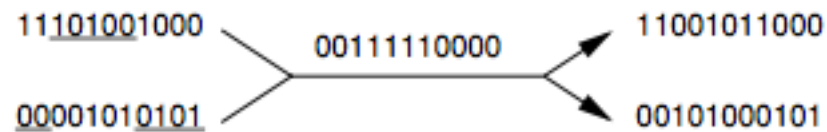
- Will use an **evaluation function, $g(c)$**
 - Which is dependent on desirable properties of the solutions (defining this is often tricky)
- Use $g(c)$ to define a **fitness function**

<i>Initial strings</i>	<i>Crossover Mask</i>	<i>Offspring</i>
------------------------	-----------------------	------------------

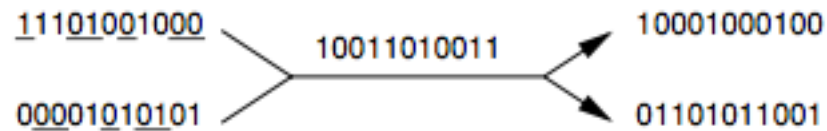
Single-point crossover:



Two-point crossover:



Uniform crossover:



Point mutation:



**UNIVERSITY
OF MALAYA**

The Leader in Research & Innovation

Calculating Fitness

- Use a **fitness function**
 - To assign a probability of each individual being used in the mating process for the next generation
- Usually use $\text{fitness}(c) = g(c)/A$
 - Where A = average of $g(c)$ over the entire population
- Other constructions are possible

Using the Fitness Function

- Use fitness to produce an intermediate population IP
 - From which mating will occur
- Integer part of the fitness value is:
 - The number of copies of individuals guaranteed to go in IP
- Fractional part is:
 - A probability that an extra copy will go into IP
 - Use a probabilistic function (roll dice) to determine whether
 - The extra copies are added to the IP

Examples of Selection

- Example #1
 - Suppose the average of g over the population is 17
 - And for a particular individual c_0 , $g(c_0) = 25$
 - $\text{Fitness}(c_0) = 25/17 = 1.47$
 - Then 1 copy of c_0 is guaranteed to go into IP
 - And the probability of another copy going in is 0.47
- Example #2
 - Suppose $g(c_1) = 14$ $\text{Fitness}(C0) = 14 / 17 = 0.82$
 - No copy of c_1 is guaranteed to go into IP because the first digit is 0;
 - The probability of one copy going in is $14/17 = 0.82$

The Mating Process

- Pairs from the Intermediate Population
 - Are chosen randomly
 - Could be the same individual twice
- They are combined with a probability p_c
 - Their offspring are produced by recombination
 - See later slides
 - Offspring go into the next generation population
- The process is repeated until
 - The next generation contains the required number of individuals for the next population
 - Often this is taken as the current population size
 - Which keeps the population size constant (not a bad idea)

Analogy with Natural Evolution

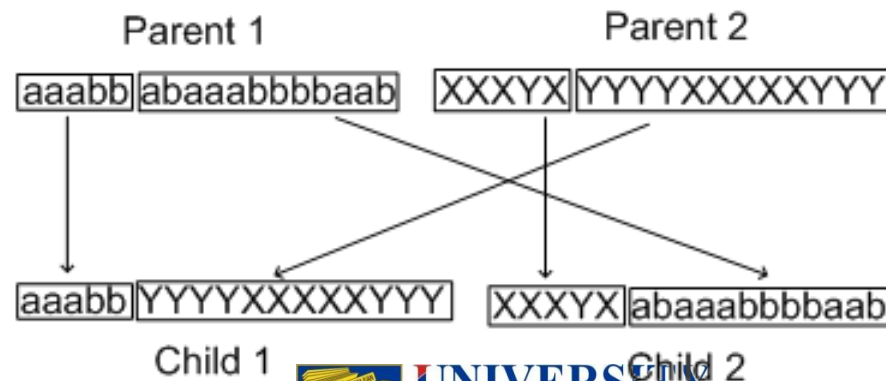
- Analogy holds:
 - Some very fit individuals may be unlucky, and:
 - (a) not find a partner
 - (b) not be able to reproduce with the partner they find
 - Some less fit individuals may be lucky and:
 - Find a partner and produce hundreds of offspring!
- Analogy breaks:
 - There is no notion of sexes
 - Although there are some types of asexual worms, etc.,
 - Individuals can mate with themselves
 - Although cells reproduce alone...

The Recombination Process

- The population will only evolve to be better
 - If best parts of the best individuals are combined
- Crossover techniques are used
 - Which take material from each individual
 - And adds it to “offspring” individuals
- Mutation techniques are used
 - Which randomly alter the offspring
 - Good for getting out of local maxima

One-point crossover

- Randomly choose a single point in both individuals
 - Both have the same point
 - Split into $LHS_1 + RHS_1$ and $LHS_2 + RHS_2$
- Generate two offspring from the combinations
 - Offspring 1: $LHS_1 + RHS_2$
 - Offspring 2: $LHS_2 + RHS_1$
- Example: (X,Y,a,b are all ones and zeros)

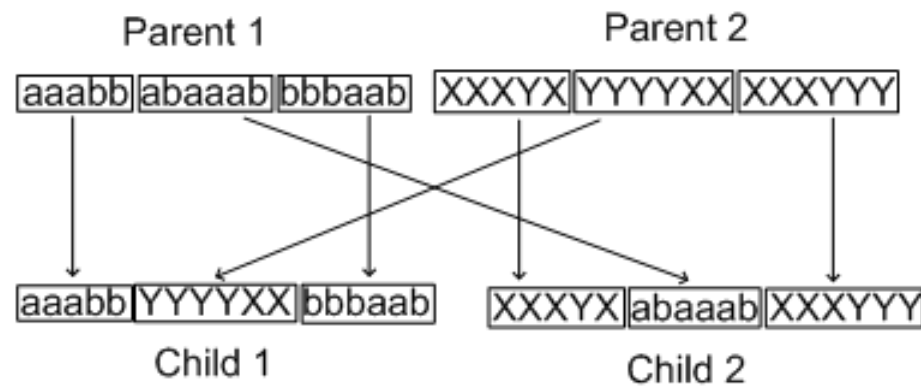


UNIVERSITY
OF MALAYA

The Leader in Research & Innovation

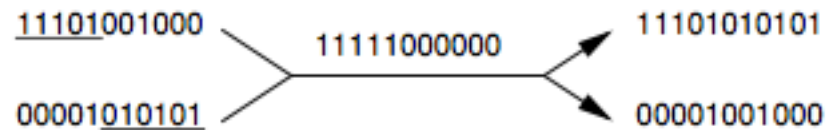
Two-point Crossover

- Two points are chosen in the strings
- The material falling between the two points – is swapped in the string for the two offspring
- Example:

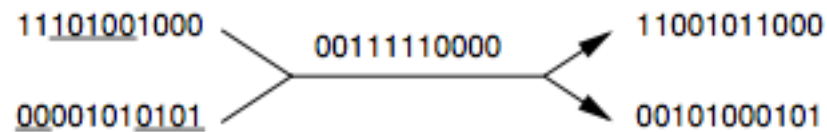


<i>Initial strings</i>	<i>Crossover Mask</i>	<i>Offspring</i>
------------------------	-----------------------	------------------

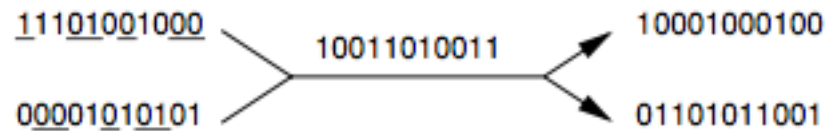
Single-point crossover:



Two-point crossover:



Uniform crossover:



Point mutation:



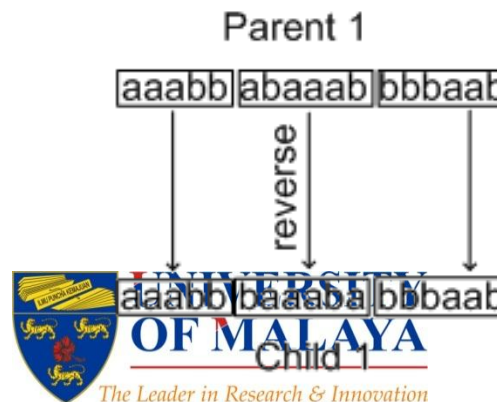
**UNIVERSITY
OF MALAYA**

The Leader in Research & Innovation

Inversion

- Another possible operator to mix things up
 - Takes the material from only on parent
- Randomly chooses two points
 - Takes the segment in-between the points
 - Reverses it in the offspring

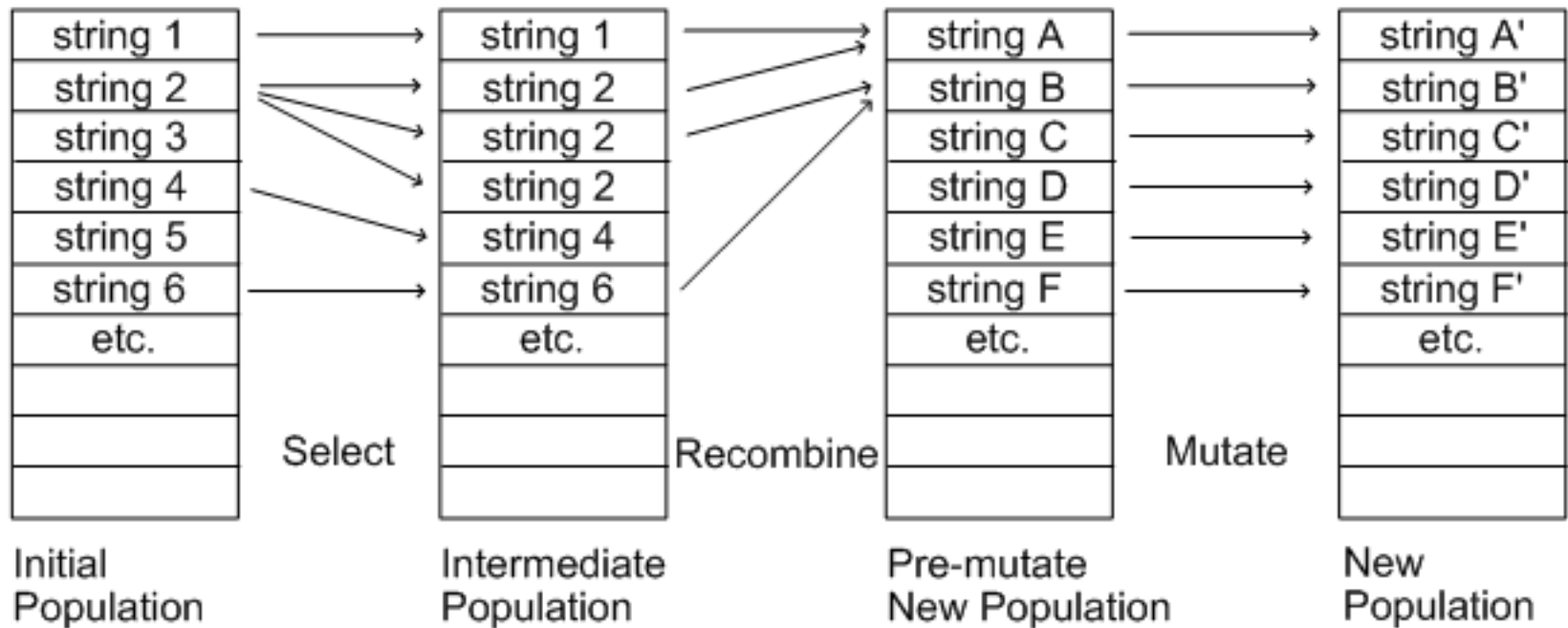
Example:



The Mutation Process

- Recombination keeps large strings the same
 - Not just randomly jumbling up the bit-strings (important point)
 - Produces a large range of possible solutions
 - But might get into a local maxima
- Random mutation
 - Each bit in every offspring produced
 - Is “flipped” from 0 to 1 or vice versa
 - With a given probability (usually pretty small $< 1\%$)
 - May help avoid local maxima
 - In natural evolution mutation is almost always deleterious
- Could use probability distribution
 - To protect the best individuals in population
 - But is this good for avoiding local maxima?

Schematic Overview for Producing the Next Generation



Selection Process

Fitness proportionate selection:

$$\Pr(h_i) = \frac{Fitness(h_i)}{\sum_{j=1}^p Fitness(h_j)}$$

... can lead to *crowding*

Tournament selection:

- Pick h_1, h_2 at random with uniform prob.
- With probability p , select the more fit.

Rank selection:

- Sort all hypotheses by fitness
- Prob of selection is proportional to rank

The Termination Check

- Termination may involve testing whether an individual solves the problem to a good enough standard
 - Not necessarily having found a definitive answer
- Alternatively, termination may occur after a fixed time
 - Or after a fixed number of generations
- Note that the best individual in a population
 - May not be as good as the best in a previous popn
- So, your GA should:
 - Record the best from each generation
 - Output the best from all populations as the answer

First Big Problem: Representing Solutions

- Difficulty 1:
 - Encoding solutions as bit strings in the first place
 - ASCII strings require 8 bits (=1 byte) per letter
 - Solutions can become very long
 - So that evolution takes many cycles to converge
- Difficulty 2:
 - Avoiding redundancy
 - Many bit strings produced by recombination and mutation will not represent possible solutions at all
 - How do we evaluate such possibilities (obviously bad???)
 - Situation is better when solutions are continuous
 - Over a set of real-valued numbers or integers
 - Situation is worse when there is a finite number of solutions

Second Big Problem: Evaluation Functions

- Fitness function is not to be confused
 - With the evaluation function
- Evaluation function must, if possible:
 - Return a real-valued number
 - Be quick to calculate (this will be done many, many times)
 - Distinguish between different individuals
- Problem when populations have evolved
 - All individuals score highly with respect to fitness
 - So we may have to do some fixing to spread them out
 - Otherwise they all have roughly equal chance of reproducing
 - And evolution will have stopped

An Example Application to Transportation System Design

- Taken from the ACM Student Magazine
 - Undergraduate project of Ricardo Hoar & Joanne Penner
- Vehicles on a road system
 - Modelled as individual agents using ANT technology
 - (Also an AI technique)
- Want to increase traffic flow
- Uses a GA approach to evolve solutions to:
 - The problem of timing traffic lights
 - Optimal solutions only known for very simple road systems
- Details not given about bit-string representation
 - But traffic light switching times are real-valued numbers over a continuous space

Transport Evaluation Function

- Traffic flow is increased if:
 - The average time a car is waiting is decreased
 - Alternatively, the total waiting time is decreased
- They used this evaluation function:

$$\text{wait} = \sum_{i \in \text{Cars}} \frac{w_i}{d_i + w_i}$$

Figure 3b: Fitness function for sequence evaluation
where w_i denotes total waiting time and d_i denotes total driving time for car i .

- W_i is the total waiting time for car i and d_i is the total driving time for car i .

Results

- Reduction in waiting times for a simple road:

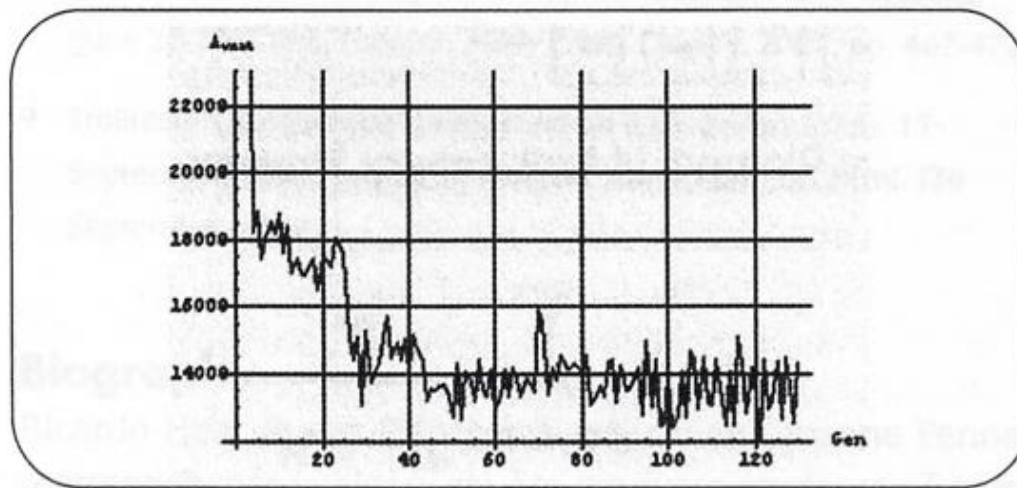


Figure 4b: Improvement in fitness over 130 generations.

- Solutions similar to (human) established solutions
 - Verification that the system is acting rationally

Results

- Reduction in waiting times for a more complicated road system (not too complicated, though)

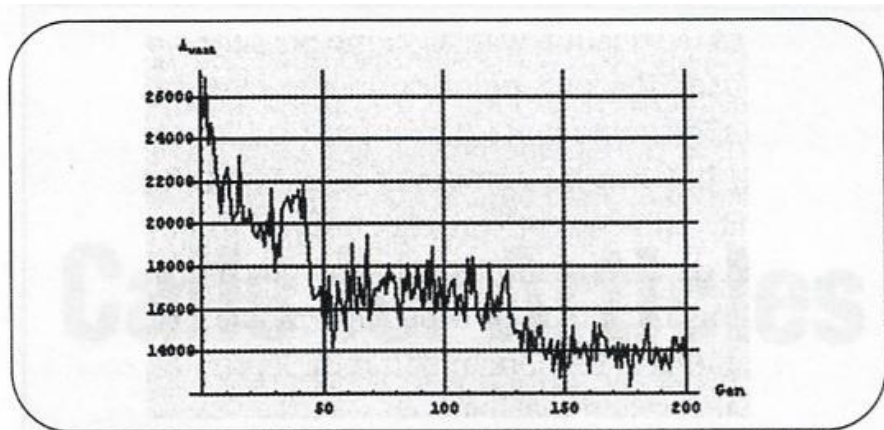
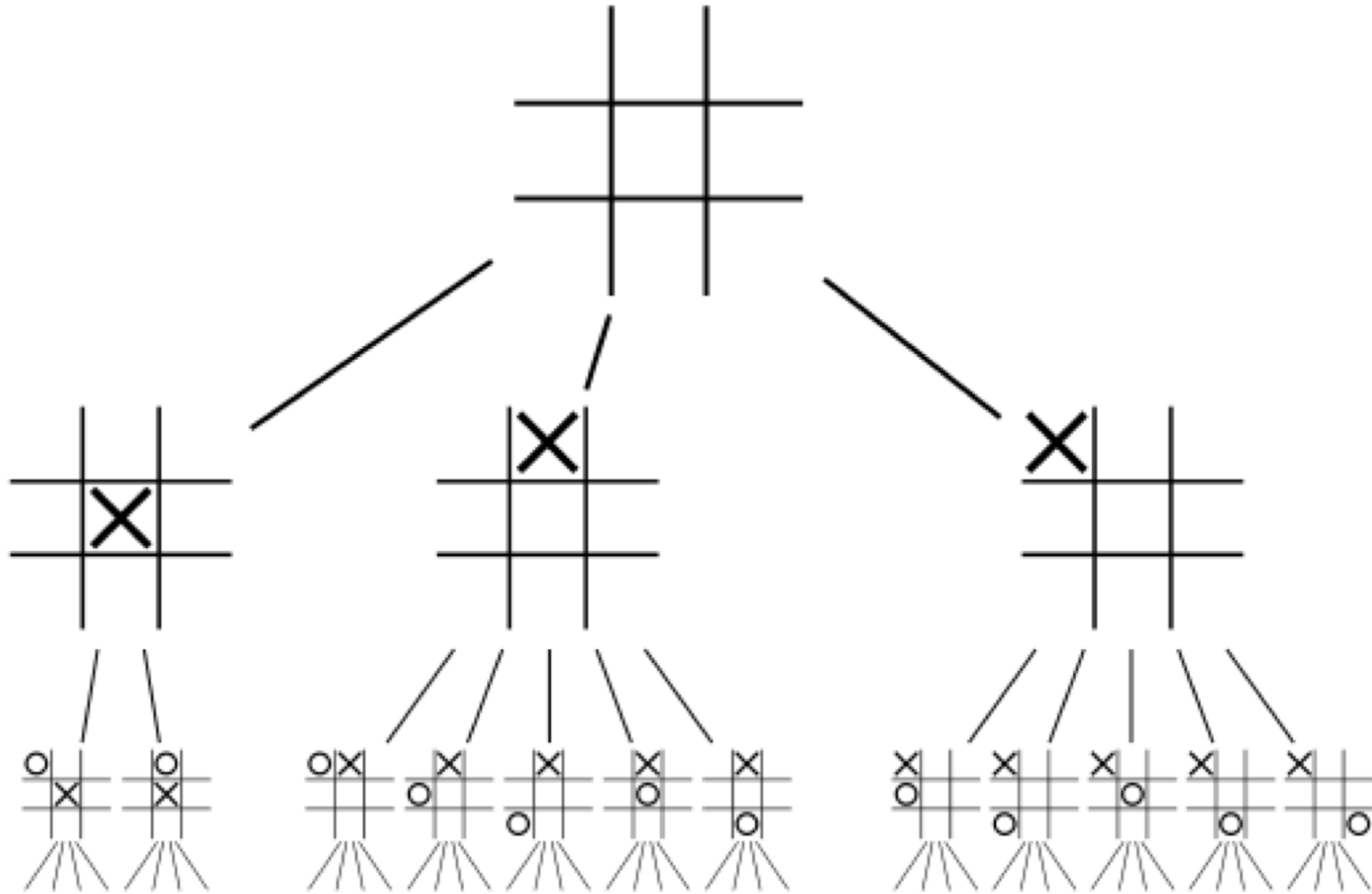


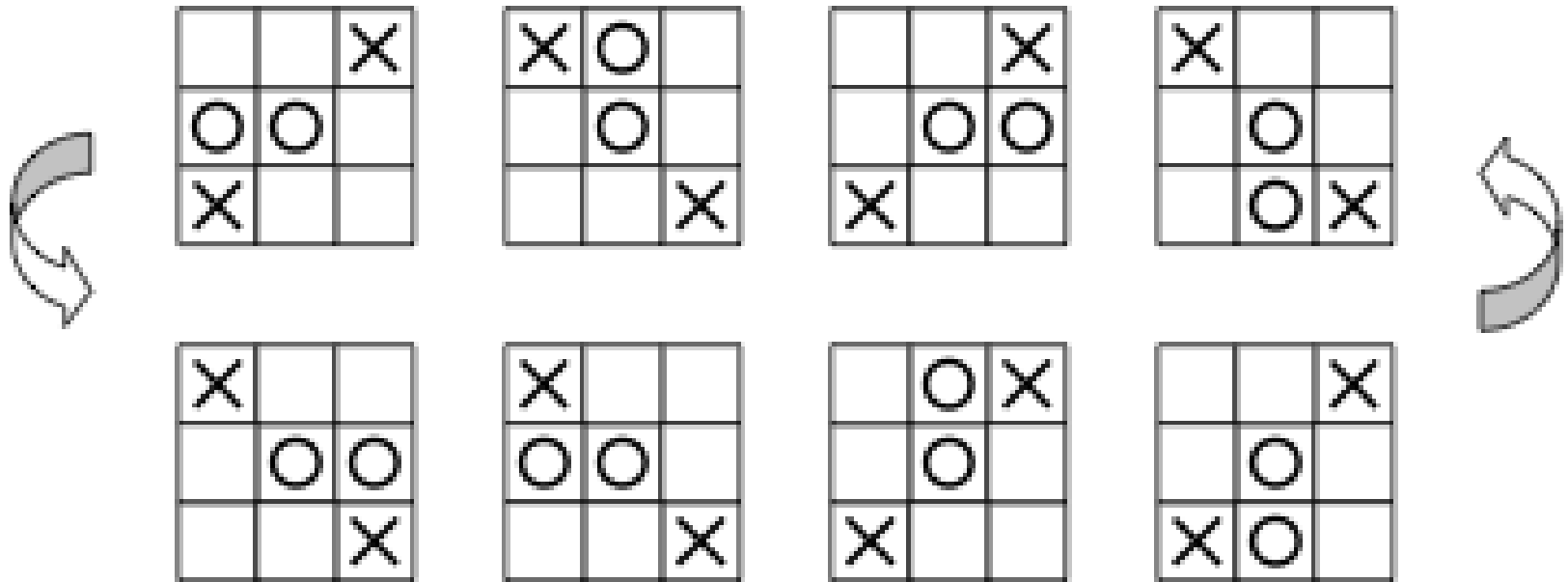
Figure 5b: Improvement in fitness over 200 generations.

- Roughly 50% decrease in waiting times
 - In both experiments

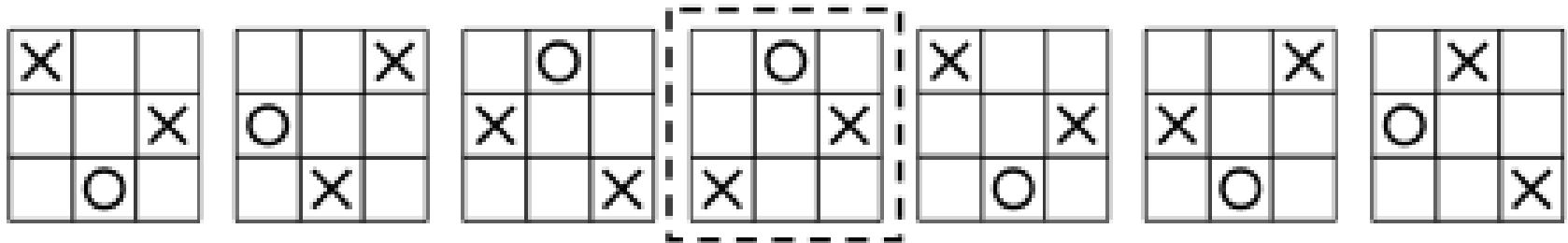
E.g. Learning tic-tac-toe using Genetic Algorithm



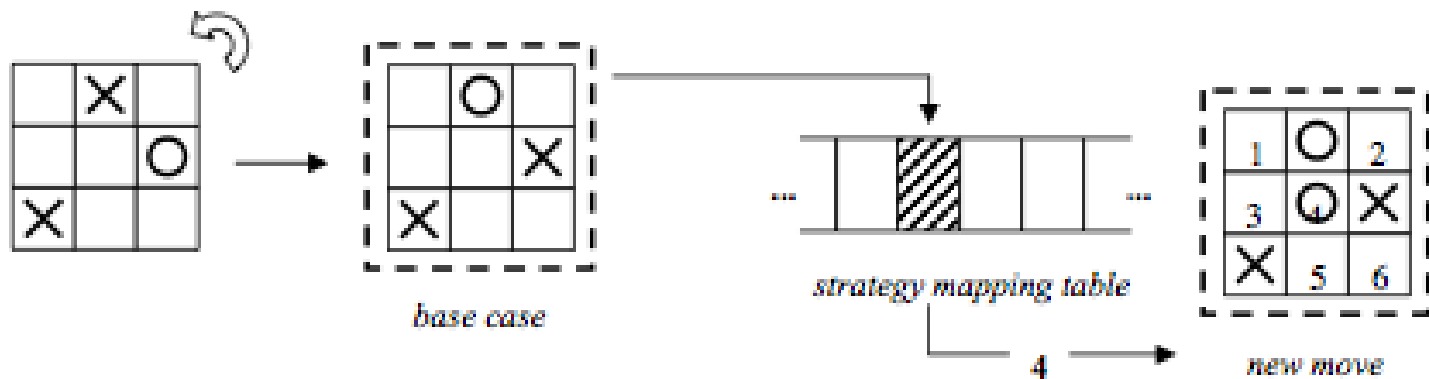
Search Space



Search Space



base case



Using Genetic Algorithms

- Russell and Norvig:
 - If you can answer four questions:
 - What is the fitness [evaluation] function?
 - How is an individual represented?
 - How are individuals selected?
 - How do individuals reproduce?
 - Then why not try GAs for your problem?
 - Similar to neural networks and CSPs (might not be the best way to proceed, but it is quick and easy to get going)